Autor:
Ronie Martínez

Python y sus librerías fundamentales

Introducción

Python es un lenguaje de programación interpretado bastante popular en entornos de ciencias de datos y aprendizaje automático, debido a su simplicidad y escalabilidad. Gracias a que es *open-source*, cuenta con diversas librerías y *frameworks* desarrollados por la comunidad, los cuales están enfocados a manipulación, modelado y gestión de datos. En el presente material se explorarán las librerías más importantes con respecto a la ciencia de datos.

Palabras clave

Python, librerías, ciencia de datos, aprendizaje automático y profundo.

Reto

La cantidad y diversidad de datos en los ambientes de trabajo actuales representan un reto importante. La ciencia de datos se presenta como un campo académico multidisciplinario que explora, organiza y modela estos datos para extraer información valiosa, inferir y asistir en la toma de decisiones.

Desarrollo

Python™

Python es un **lenguaje de programación** de alto nivel y de propósito general, creado por Guido van Rossum en 1991. Según su web oficial (Python TM, 2023): "Python te permite trabajar más rápidamente e integrar tus sistemas de manera más efectiva". Debido a su **simplicidad**, sintaxis fácil de usar y comprender, y

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

una amplia variedad de **librerías** y marcos de trabajo, ha ganado gran popularidad en diversos campos, incluidos: desarrollo web y de *software*, **análisis de datos**, **aprendizaje automático** e **inteligencia artificial**, computación científica, automatización, entre otros.

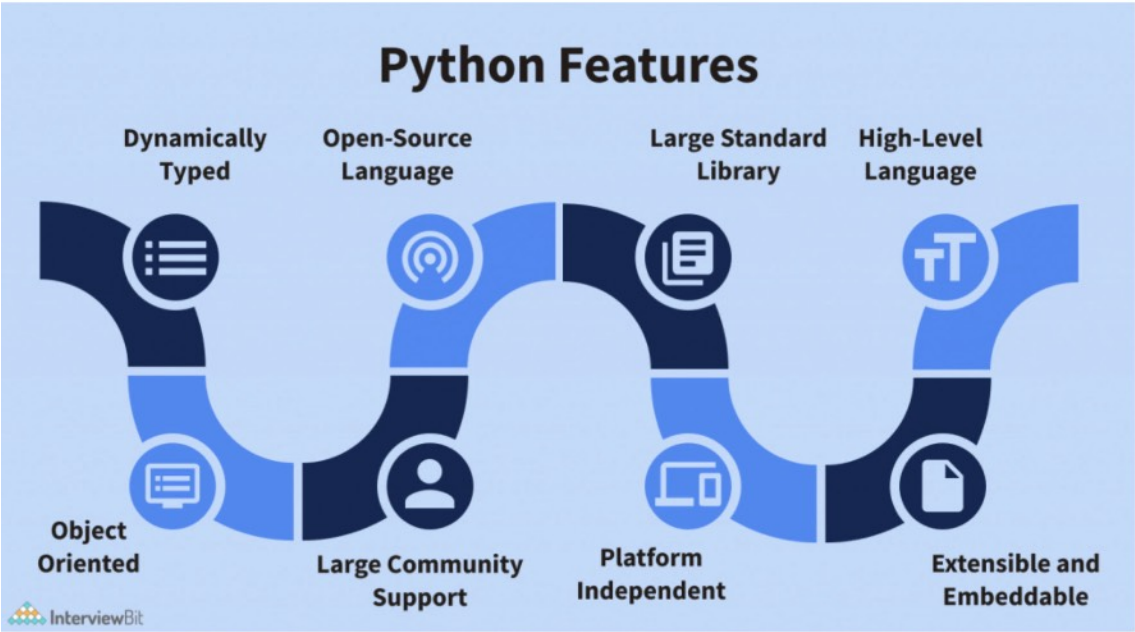
Algunas características clave (InterviewBit, 2023; MKS075, 2019) son:

- **Legibilidad** (*readability*): Hace hincapié en el código limpio y fácil de leer, lo que lo hace accesible tanto para principiantes como para desarrolladores experimentados. Utiliza indentación (*whitespace*) para definir bloques de código, lo que impone un estilo de codificación claro y consistente.
- **Lenguaje de alto nivel** (*high-level language*): Abstrae muchos detalles de programación de bajo nivel, permitiendo a los desarrolladores centrarse en la solución de problemas en lugar de las complejidades de la gestión de la memoria y otras tareas de bajos niveles.
- **Interpretado** (*interpreted*): Al ser un lenguaje interpretado no se necesita compilar el código, lo que significa que el código puede ser ejecutado línea por línea, por lo que es adecuado para el desarrollo interactivo y el uso de secuencia de comandos (*scripting*). Además, permite una fácil depuración (*debugging*) y portabilidad.
- **Programación orientada a objetos – POO** (*object-oriented language*): Soporta programación orientada a objetos, incluyendo conceptos de clases, encapsulación de objetos y herencia.
- **Multiplataforma** (*cross-platform*): Está disponible en varias plataformas (Windows, macOS, Linux) y se utiliza ampliamente en el desarrollo multiplataforma.
- **Amplia biblioteca** (*extensive standard library*): Viene con una extensa biblioteca estándar que proporciona módulos y paquetes para una amplia gama de tareas, desde el manejo de archivos hasta protocolos de Internet. Dentro del vasto ecosistema de librerías y marcos de trabajo (*frameworks*), están NumPy, Pandas, Matplotlib, Django, Flask, TensorFlow y PyTorch.
- **Gratis y de código abierto** (*open source*): Lo que significa que es libre de usar, modificar y distribuir. La comunidad de Python contribuye activamente a su desarrollo y mantenimiento, proporcionando una amplia documentación, tutoriales y recursos para los desarrolladores.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

- **Extensible e incrustable** (*extensible and embeddable*): Se puede ampliar con otros lenguajes de programación, como java, C y C++, permitiendo mayor funcionalidad en su código.
- **Interfaz de usuario gráfica GUI** (*Graphical User Interface*): Soporta la programación de GUI a través de varias librerías, como PyQt5, PyQt4, wxPython y Tk.

ILUSTRACIÓN 1. CARACTERÍSTICAS DE PYTHON. TOMADO DE (INTERVIEWBIT, 2023)



Ciencia de Datos (Data Science)

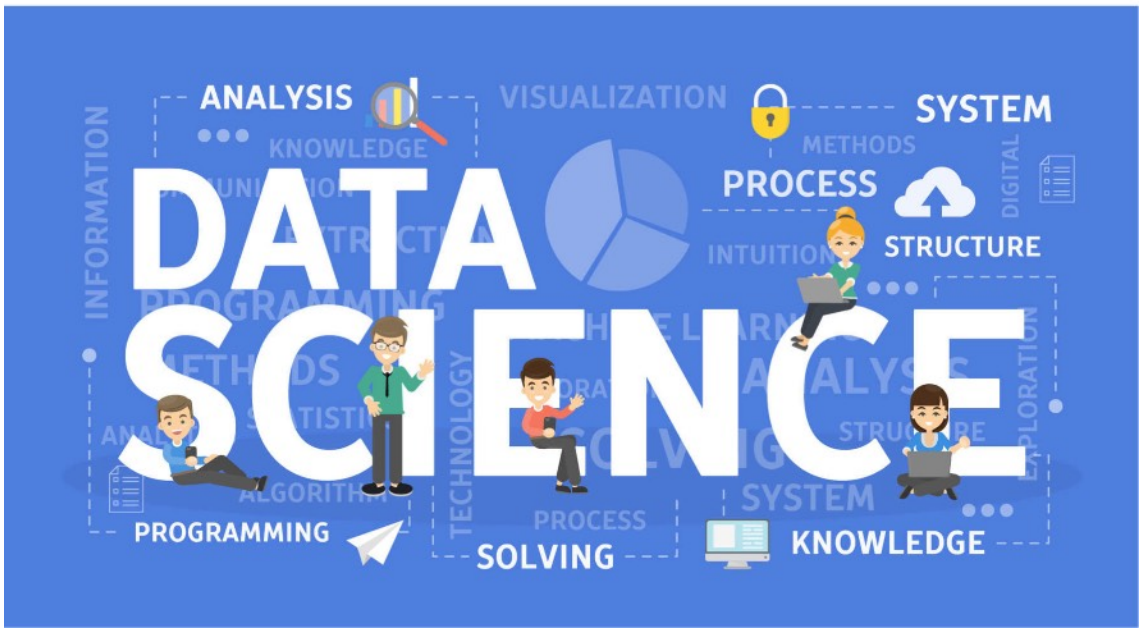
Entre 2011 y 2013 (apenas 2 años), el volumen mundial de datos se multiplicó por 9. Al final de 2020, el volumen total de datos a escala mundial alcanzó los 44 zettabytes en comparación a los menos de 5 zettabytes en 2013. De esta manera, los datos se han convertido en un recurso muy valioso, por lo que algunos lo han denominado como "el petróleo del siglo XXI" (Dst Team, 2021).

La ciencia de datos es un campo académico **multidisciplinario** que abarca tres áreas distintas y superpuestas: las habilidades de un **estadista** que puede modelar y resumir conjuntos de datos (que están creciendo cada vez más); las capacidades de un **científico informático** que puede diseñar y utilizar algoritmos para almacenar, procesar y visualizar estos datos de manera eficiente; y la **experiencia** del dominio (lo que se podría pensar como una

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

formación "clásica" en un tema) necesaria tanto para formular las preguntas correctas como para poner sus respuestas en contexto (VanderPlas, 2016). Este conjunto de habilidades tiene una amplia gama de aplicaciones, por ejemplo: informar los resultados de elecciones, predecir el rendimiento de acciones, optimizar los clics en anuncios en línea, identificar microorganismos en fotografías del microscopio, buscar nuevas clases de objetos astronómicos, detección de anomalías o fraude, sistemas de recomendación, o trabajar con datos en cualquier otro campo, básicamente cualquier campo en el que se puedan extraer datos. En pocas palabras, la ciencia de datos es el proceso de usar datos para encontrar soluciones o predecir resultados a un problema planteado mediante análisis descriptivo y predictivo.

ILUSTRACIÓN 2. ¿QUÉ ES LA CIENCIA DE DATOS? TOMADO DE (WHITE, 2021)



La utilidad de Python para la ciencia de datos deriva principalmente del gran ecosistema activo de paquetes de terceros: **NumPy** para la manipulación de datos homogéneos basados en arreglos (*array*), **Pandas** para manipular datos heterogéneos y etiquetados, **SciPy** para tareas comunes de computación científica, **Matplotlib** para visualizaciones de calidad al nivel de publicación, **IPython** para ejecución interactiva y compartir código, **Scikit-Learn** para el aprendizaje automático, etc.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

IPython (Interactive Python)

Python interactivo proporciona un entorno integral para la computación **interactiva** y **exploratoria**. Dentro de sus múltiples características se encuentran:

Acceso a la documentación con '**?**': todo objeto en Python contiene la referencia a una cadena de texto (*string*) conocida como *docstring*, en donde se encuentra la documentación en forma de resumen del objeto y cómo usarlo. Es similar o un atajo de la función **help()**.

Acceso al código fuente con '**??**': excepto para objetos que no son implementados en Python (por ejemplo, *C*). En este caso, el resultado es el mismo que usar '**?**'.

Exploración de módulos y autocompletación con **<TAB>**: al escribir el nombre del objeto, seguido de un punto '**.**' y **<TAB>** se despliega una lista completa de todos los atributos disponibles. Mientras que si se empieza a escribir el nombre y se presiona **<TAB>** se buscarán los atributos y métodos que coincidan, reduciendo las opciones o autocompletando directamente (cuando sea coincidencia única). Python no tiene distinción entre atributos públicos/externos y privados/internos (por convención, los nombres son precedidos de guion bajo '_').

Wildcard matching con '*****': busca coincidencias de la cadena de texto parcial deseada, independientemente de su ubicación en la cadena de texto total.

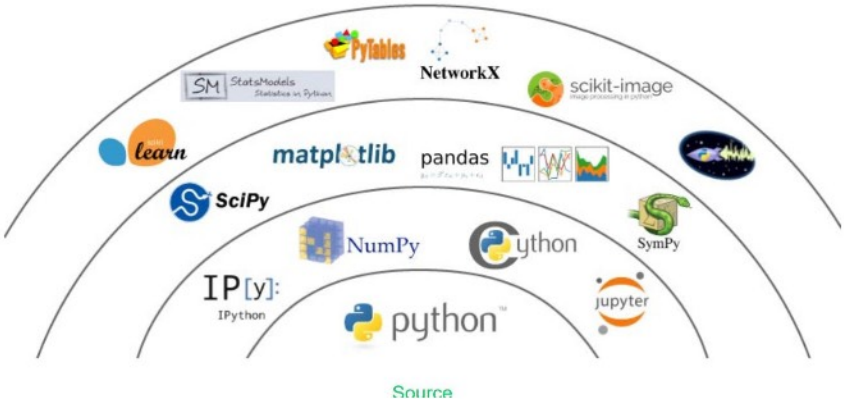
Ejecutar código externo con '**%run**': se pueden ejecutar archivos o scripts con el comando mencionado, seguido del nombre del archivo. Una vez que se haya ejecutado el archivo, las funciones internas contenidas en dicho archivo, se vuelven disponibles en la sesión actual.

Calcular el tiempo de ejecución con '**%timeit**': para una línea de código, mientras que para un bloque se utiliza '**%%timeit**'.

Acceso al historial de entrada y salida: se guardan automáticamente en las variables '**In**' y '**Out**', y se pueden acceder a sus elementos mediante indexación numérica. Adicionalmente, las salidas previas se guardan también en la variable '_' (guion bajo), aumentando el número de guiones bajos según más se quiera retroceder en el historial. De igual manera, para acceder a las entradas previas se puede usar '**%history**'.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

ILUSTRACIÓN 3. OVERVIEW DEL ECOSISTEMA PYTHON. TOMADO DE (ALEX J BOWERS [@ALEX_J_BOWERS], 2019)



NumPy (Numerical Python)

Los conjuntos de datos (*datasets*) pueden provenir de una amplia **variedad** de **fuentes** y una amplia gama de **formatos**, incluyendo colecciones de **documentos**, colecciones de **imágenes**, colecciones de clips de **sonido**, colecciones de medidas **numéricas**, o cualquier otra cosa. A pesar de esta aparente heterogeneidad, NumPy (NumPy, 2023) ayuda a pensar en todos los datos fundamentalmente como conjuntos o **arreglos de números**. Por ejemplo, las imágenes (especialmente las imágenes digitales), pueden considerarse simplemente arreglos bidimensionales de números que representan el brillo de los píxeles en toda la zona. Los clips de sonido pueden ser pensados como arreglos unidimensionales de intensidad versus tiempo. El texto puede ser convertido de varias maneras en representaciones numéricas, por ejemplo, cifras binarias que representan la frecuencia de ciertas palabras o pares de palabras. Independientemente de lo que sean los datos, el primer paso para hacerlos analizables será transformarlos en arreglos numéricos.

Numpy es una librería de Python para computación científica que provee soporte eficiente para almacenar y operar *buffers* de datos densos y multidimensionales, que se vuelven más eficientes a medida que los arreglos aumentan de tamaño. De esta manera, se ha convertido en una librería indispensable en aplicaciones científicas y de ingeniaría, incluyendo procesamiento de señales e imágenes, álgebra lineal y *machine learning*, es decir, casi cualquier entorno de trabajo que involucre el manejo de datos.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

A diferencia de las listas en Python, NumPy está delimitado a arreglos que contienen el mismo tipo de datos. Si los tipos de datos no coinciden, NumPy actualiza automáticamente (si es posible) a un tipo de dato que englobe todo, por ejemplo, si se tiene enteros (*int*) y flotantes (*float*), los enteros se convertirán a flotantes. Cuando se define el arreglo, también está la opción de definir el tipo de dato.

Dentro de las categorías esenciales para manejo de datos en NumPy están las que se enfocan en atributos, indexado, particionado, reorganizado, combinado y separación de arreglos (VanderPlas, 2016), y serán ahondadas en las lecturas de profundización posteriores.

Para muchos tipos de operaciones, NumPy proporciona una interfaz conveniente para este tipo de rutina compilada y escrita estadísticamente, lo que provee una ejecución rápida y optimizada, ya que internamente se hace un procesamiento con funciones propias de NumPy (*universal functions – ufuncs*) pero que son completamente transparentes al usuario. Esto es conocido como una operación vectorizada y se puede lograr simplemente realizando una operación sobre el arreglo, la cual se aplicará a cada elemento, ajustándose a las dimensiones necesarias (siempre y cuando sea posible).

Pandas

Pandas (Pandas, 2023) es un paquete más actual, creado sobre NumPy, y que proporciona una implementación eficiente de un DataFrame. Los DataFrames son esencialmente **matrices multidimensionales** con **etiquetas** de filas y columnas, a menudo con tipos de datos heterogéneas y/o datos faltantes. Además de ofrecer una cómoda interfaz de almacenamiento para datos etiquetados, Pandas implementa una serie de poderosas operaciones de datos familiares para usuarios que manejan tanto de marcos de trabajo de bases de datos como de programas de hojas de cálculo (valores separados por comas, JSON, Parquet, SQL tablas de bases de datos o consultas, y Microsoft Excel).

La estructura de datos '*ndarray*' de NumPy proporciona características esenciales para la **limpieza** y **organización** de datos, y si bien sirve muy bien para este propósito, sus **limitaciones** se vuelven claras cuando necesitamos más flexibilidad (agregar etiquetas a los datos, trabajar con datos faltantes, etc.) y cuando intentamos operaciones que no se asignan bien en el *element-wise broadcasting* (agrupaciones, pivotes, etc.); esto es esencial para analizar datos menos estructurados disponibles en muchas aplicaciones/problemas del mundo real. Pandas, y en particular sus objetos Series y DataFrame, se basa en la

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

estructura de arreglos NumPy y proporciona acceso eficiente a este tipo de tareas de "gestión de datos" que ocupan gran parte del tiempo de un científico de datos (VanderPlas, 2016). Los objetos se describen brevemente a continuación:

- **Series:** Arreglo unidimensional de datos indexados. Contiene la secuencia de valores y una secuencia de índices adicional (definido explícitamente), y a diferencia de NumPy, se pueden acceder a los elementos deseados mediante indexación numérica y no numérica (*strings*).
- **DataFrame:** Arreglo bidimensional con índices flexibles en filas y columnas. Se puede pensar también como una secuencia de objetos Series alineados (que comparten el mismo índice), en donde se llenan los datos faltantes con valores NaN (*not a number*) por defecto.
- **Index:** Los objetos Series y DataFrame contienen un índice (*index*) explícito que permite referenciar y modificar los datos. El objeto Index es una estructura y puede ser pensada como un arreglo inmutable o como un conjunto ordenado (técnicamente múltiple, ya que los objetos Index pueden tener valores repetidos).

Matplotlib y Seaborn

Matplotlib es una librería de visualización y gráfica de datos multiplataforma construida sobre arreglos NumPy y diseñada para funcionar con SciPy. Fue concebido en 2002 por John Hunter, originalmente como un parche para IPython para permitir el **graficado interactivo** estilo MATLAB. El creador de IPython, Fernando Pérez, estaba en ese momento terminando su doctorado y le comunicó a John que no tendría tiempo para revisar el parche hasta dentro de varios meses. De esta manera, John tomó esto como una oportunidad y así nació el paquete Matplotlib, cuya versión 0.1 se lanzó en 2003. Recibió un impulso inicial cuando fue adoptado como el paquete de graficado elegido por el *Space Telescope Science Institute* (grupo detrás del Telescopio Hubble), que apoyó financieramente el desarrollo de Matplotlib y amplió enormemente sus capacidades. Específicamente, el método más usado de Matplotlib es Pyplot.

Funciona con una **interfaz del estilo de MATLAB** (ya que originalmente se creó la librería como una alternativa), es decir, crear la figura e ir añadiendo elementos (gráficas, subgráficas, atributos, etc.), o también funciona con una **interfaz orientada a objetos**, la cual generalmente se usa en situaciones más complejas, en donde se quiera tener un mejor control sobre los elementos de la

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

figura. Dentro de los tipos de gráficos que se pueden realizar con esta librería están: líneas, barras, histogramas, dispersión, circulares/pastel, áreas, 3D, etc.

La librería Seaborn nace como una actualización de Matplotlib y su método **set()** sobreescribe los parámetros de Matplotlib por defecto para obtener una presentación superior y con un *script* más simple.

Scikit-Learn

El aprendizaje automático (*machine learning – ML*), como subgrupo de la inteligencia artificial - IA, es el principal medio por el cual la ciencia de datos se ha expandido a muchos campos, ya que es en donde las **habilidades computacionales** y **algorítmicas** se encuentran con el **pensamiento estadístico**, y el resultado es una colección de acercamientos para la **inferencia**, **aprendizaje** y la **exploración** a partir de los **datos**. Existen dos grandes grupos cuando se habla de ML: aprendizaje supervisado y no supervisado, cuya principal diferencia es tener o no datos etiquetados. A partir de estos dos grandes grupos se generan nuevos subgrupos, por ejemplo: aprendizaje semisupervisado, aprendizaje por refuerzo (*reinforcement learning*) e inferencia transductiva (*transduction*).

Scikit-Learn es una librería de Python que provee implementaciones eficientes de una amplia gama de algoritmos de ML, por ejemplo: regresión lineal, naive bayes, árboles de decisión, máquinas de soporte vectorial, agrupamiento (*clustering*), etc. Si bien, el proceso de diseño e implementación de un algoritmo de ML puede ser analizado más extensamente, básicamente se resume en los siguientes pasos:

- Escoger el modelo de ML a usar e importar la clase correspondiente desde Scikit-Learn.
- Escoger e inicializar los hiperparámetros con los valores deseados.
- Organizar los datos en matriz de características (*features matrix*) y vector objetivo/respuesta (*target vector*), respetando los formatos.
- Entrenar el modelo mediante el método **fit()**.
- Aplicar el modelo a datos nuevos usando el método **predict()**.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Pytorch, Keras y TensorFlow

Más allá del ML tradicional, existen ocasiones en las que estos algoritmos no son suficientes para los datos que se tiene, ya sea por su estructura o por su cantidad. Aunque no son nuevas, en los últimos años se han popularizado las redes neuronales, tanto las básicas como las que tienen topologías complejas, conocidos como algoritmos de aprendizaje profundo (*deep learning – DL*). PyTorch, Keras y TensorFlow son marcos de DL populares que se utilizan para crear y entrenar modelos (Brownlee, 2016). A continuación, una breve descripción:

PyTorch: Librería que proporciona una variedad de algoritmos de aprendizaje supervisados y no supervisados. Fue diseñado desde los primeros principios para admitir un estilo de programación imperativo y orientado a Python que admita el código como modelo, facilite la depuración y sea coherente con otras bibliotecas informáticas científicas populares. PyTorch es conocido por su flexibilidad y facilidad de uso, lo que lo convierte en una opción popular tanto para investigadores como para desarrolladores. También es conocido por su uso eficiente de aceleradores de hardware como las GPU.

Keras: API (*Application Programming Interface*) de redes neuronales de alto nivel, escrita en Python y capaz de ejecutarse sobre TensorFlow, CNTK o Theano. Fue diseñado para permitir una experimentación rápida con redes neuronales profundas y se centra en su facilidad de usar, modular y ser extensible.

TensorFlow: Librería de aprendizaje automático de código abierto desarrollada por *Google Brain Team*. Está diseñado para ser flexible, escalable y listo para producción, y admite una amplia gama de tareas de aprendizaje automático, incluido el aprendizaje profundo, el aprendizaje por refuerzo y el modelado probabilístico. TensorFlow es conocido por su alto rendimiento y escalabilidad, lo que lo convierte en una opción popular para aplicaciones de aprendizaje automático a gran escala.

Conclusiones

Al final de 2020, el volumen total de datos a escala mundial alcanzó los 44 zettabytes en comparación a los menos de 5 zettabytes en 2013. Debido a este crecimiento en cantidad de datos, estos se han convertido en un recurso muy valioso, denominados por algunos como “el petróleo del siglo XXI”.

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

De esta manera, la cantidad y diversidad de datos en los ambientes de trabajo actuales representan un reto importante a ser considerado. La ciencia de datos se presenta como un campo académico multidisciplinario que abarca tres áreas distintas y superpuestas: las habilidades de un estadista que puede modelar y resumir conjuntos de datos; las capacidades de un científico informático que puede diseñar y utilizar algoritmos para almacenar, procesar y visualizar estos datos de manera eficiente; y la experiencia del dominio necesaria tanto para formular las preguntas correctas como para poner sus respuestas en contexto.

Python al ser un lenguaje de programación interpretado, *open-source*, simple y escalable, se ha convertido en una herramienta bastante popular en aplicaciones de ciencias de datos, aprendizaje automático y profundo, con el objetivo de manipular, modelar y gestionar datos. Gracias a su amplia comunidad, cuenta con varias librerías y *frameworks* que son de gran utilidad al momento de realizar estas actividades. En el presente material se exploraron las librerías más importantes con respecto a la ciencia de datos, entre los que están: IPython para interactividad y revisar documentación de las funciones internas, NumPy y Pandas para manipulación y organización, Matplotlib y seaborn para visualización y graficado, Scikit-Learn para modelado de algoritmos tradicionales y relativamente sencillo de aprendizaje automático, y Pytorch, Keras y Tensorflow para modelado de algoritmos más complejos y robustos de aprendizaje profundo. Todas estas herramientas y librerías mencionadas son propuestas y utilizadas con la finalidad de extraer información valiosa, inferir y asistir en la toma de decisiones.

Este material se presentó como una introducción al módulo denominado "Arquitectura, modelado y gestión de datos en Data Science" en donde se exploró la herramienta Python y sus librerías más importantes. En materiales posteriores se explorará la manipulación, el análisis y el procesamiento de datos, lo que engloba revisar diversas fuentes y formatos de origen y su acceso, y la limpieza, agrupación, y completado de datos.

Referencias

Alex J Bowers [@Alex_J_Bowers]. (2019, julio 11). *Great overview of the #Python ecosystem. Top 30 Python Libraries for #MachineLearning. #NumPy #Pandas #Scipy #Matplotlib #Scikit-Learn #Statsmodels https://morioh.com/p/a42cb68ff2b5/top-25-python-libraries-for-machine-learning https://t.co/IsRMiyjUra* [Tweet]. Twitter. https://twitter.com/Alex_J_Bowers/status/1149227896877305856

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.

Brownlee, J. (2016). *Deep Learning With Python: Develop Deep Learning Models on Theano and TensorFlow Using Keras*. Machine Learning Mastery.

Dst Team. (2021, diciembre 27). Data Science: Definición, problemática y casos de uso. *Formation Data Science | DataScientest.com*. https://datascientest.com/es/data-science-definicion-problematica-y-casos-de-uso

InterviewBit. (2023, marzo 28). *Top 11 Features of Python You Must Know (2023)*. InterviewBit. https://www.interviewbit.com/blog/features-of-python/

MKS075. (2019, abril 3). Python Features. *GeeksforGeeks*. https://www.geeksforgeeks.org/python-features/

NumPy. (2023, septiembre 16). *NumPy*. https://numpy.org/

Pandas. (2023, octubre 26). *pandas – Python Data Analysis Library*. https://pandas.pydata.org/

Python TM. (2023, octubre 27). *Welcome to Python.org*. Python.Org. https://www.python.org/

VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media, Inc.

White, L. (2021, septiembre 25). Top 20 Data Science Projects For Beginners and Experts. *CODERSERA*. https://codersera.com/blog/data-science-projects/

Todos los derechos reservados. Prohibida la reproducción total o parcial del documento y su distribución por cualquier medio impreso o electrónico sin la autorización escrita de EIG.